

LeetCode 992: Subarrays with K Distinct Integers

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 6, 2026

TARGETED BY: GOLDMAN SACHS, D.E. SHAW, MORGAN STANLEY

1. The Most Important Rule

To solve the problem of finding the exact number of subarrays with *exactly* **K** distinct integers, we use a fundamental sliding window transformation:

$$\text{Exactly}(K) = \text{AtMost}(K) - \text{AtMost}(K - 1)$$

2. Standard Template for Counting Valid Subarrays

The core of the sliding window approach for finding "At Most K" distinct integers follows this template:

```
int l = 0;
int r = 0;
while (r < n) {
    // Include nums[r] into the window

    while (window is invalid) {
        // Remove nums[l] from the window
        l++;
    }

    // Count valid subarrays ending at r
    ans += (r - l + 1);
    r++;
}
```

Why do we add `r - l + 1` to the answer?

Because for an **at most K distinct** constraint, every starting point from **1** up to **r** creates exactly one valid subarray ending at **r**.

Visually, if the valid window is `[l, ..., r]`, the valid subarrays ending at `r` are:

`[l..r]`, `[l+1..r]`, `[l+2..r]`, ... `[r..r]`.

The total count of these is exactly **$r - l + 1$** .

3. Brute Force Approach (Intuition)

- Generate every possible subarray.
- Maintain a count of distinct elements in the current subarray.
- If the size of distinct elements $> K$, then stop extending.
- If the size of distinct elements $= K$, increment the answer.

Time Complexity: $O(N^2)$ - Too slow for large inputs.

4. Optimal C++ Implementation

Applying the `AtMost(K) - AtMost(K-1)` logic with an optimal **$O(N)$** sliding window approach.

```

class Solution {
public:
    int n;

    int atleast(vector<int>& nums, int k) {
        if(k < 0) return 0;

        int l = 0;
        int r = 0;
        int ans = 0;
        unordered_map<int, int> freq;

        while(r < n) {
            freq[nums[r]]++;

            while(freq.size() > k) {
                freq[nums[l]]--;

                if(freq[nums[l]] == 0) {
                    freq.erase(nums[l]);
                }
                l++;
            }

            ans += (r - l + 1);
            r++;
        }

        return ans;
    }

    int subarraysWithKDistinct(vector<int>& nums, int k) {
        n = nums.size();
        return atleast(nums, k) - atleast(nums, k - 1);
    }
};

```